

第一章：绪论

数据库 [DB]：长期存储在计算机、有组织、可共享的大量数据的集合。数据库中的数据按照一定的数据模型组织、描述和存储，具有较小的冗余度、较高的数据独立性和易扩展性，并可为各种用户共享。

数据库管理系统 [DBMS]：位于用户和操作系统间的数据管理系统的一层数据管理软件。用途：科学地组织和存储数据，高效地获取和维护数据。包括数据定义功能，数据组织、存储和管理，数据操纵功能，数据库的事物管理和运行管理，数据库的建立和维护功能，其他功能。

数据库系统 [DBS]：在计算机系统中引入数据库后的系统，一般由数据库、数据库管理系统 [及其开发工具]、应用系统、数据库管理员构成。目的：存储信息并支持用户检索和更新所需的信息。

数据库系统的特点：数据结构化；数据的共享性高，冗余度低，易扩大；数据独立性高；数据由 DBMS 统一管理和控制。

概念模型 实体，客观存在并可相互区别的事物称为实体。

属性，实体所具有的*一特性称为属性。

码，唯一标识实体的属性集称为码。

域，是一组具有同样数据类型的值的集合。

实体型，具有同样属性的实体必然具有的共同的特征和性质。

实体集，同一类型实体的集合称为实体集。

联系

两个实体型之间的联系 一对一联系；一对多联系；多对多联系

关系模型 关系，元组，属性，码，域，分量，关系模型

关系数据模型的操纵与完整性约束 关系数据模型的操作主要包括查询，插入，删除和更新数据。

这些操作必须满足关系完整性约束条件。关系的完整性约束条件包括三大类：实体完整性，参照完整性和用户定义的完整性。

数据库系统三级模式构造外模式，模式，模式

模式：[逻辑模式] 数据库中全体数据的逻辑构造和特征的描述，是所有用户的公共数据视图。一个数据库只有一个模式。

模式的地位：是数据库系统模式构造的中间层，与数据的物理存储细节和硬件环境无关，与具体的应用程序、开发工具及高级程序设计语言无关。

模式定义的容：数据的逻辑构造 [数据项的名字、类型、取值范围等]，数据之间的联系，数据有关的安全性、完整性要求

外模式：[子模式/用户模式] 数据库用户 [包括应用程序员和最终用户] 能够看见和使用的局部数据库和逻辑构造和特征的描述，是数据库用户的数据视图，是与*一应用有关的系统的逻辑表示。一个数据库可以有多个外模式。

外模式的地位：介于模式与应用之间

模式与外模式的关系：一对多。外模式通常是模式的子集。一个数据库可以有多个外模式。反映了不同的用户的应用需求、对待数据的方式、对数据的要求。对模式中同一数据，在外模式中的构造、类型、长度、级别等都可以不同。

外模式与应用的关系：一对多。同一外模式也可以为*一用户的多个应用系统所使用

但一个应用程序只能使用一个外模式

模式：存储模式或视图]是数据物理构造和存储方式的描述，是数据在数据库部实际存储的表示方式：

记录的存储方式 [顺序，B 树，hash 方法存储]，索引的组织方式，数据是否压缩存储，数据是否加密。
数据存储记录构造的规定，一个数据库只有一个模式

三级模式的优点：

[1] 保证数据的独立性 [模式与模式分开物理独立；外模式与模式分开逻辑独立]

[2] 简化用户窗口 [3] 有利于数据共享 [4] 利于数据的平安

[5] 数据存储由 DBMS 管理 [用户不用考虑存取路径等细节]

二级映像功能：

(1) 外模式/模式映像 [应用可扩大性]

定义外模式(局部逻辑构造)与模式(全局逻辑构造)之间的对应关系，映像定义通常包含在各自外模式的描述中，每一个外模式，数据库系统都有一个外模式/模式映像。

用途：保证数据的逻辑独立性

当模式改变时，数据库管理员修改有关的外模式/模式映像，使外模式保持不变

应用程序是依据数据的外模式编写的，从而应用程序不必修改，保证了数据与程序的逻辑独立性，简称数据的逻辑独立性。

(2) 模式/模式映像 [空间利用率，存取效率]

模式/模式映像 is 唯一的，它定义了数据全局逻辑构造与存储构造之间的对应关。数据库中模式/模式映像 is 唯一的。该映像定义通常包含在模式描述中。

用途：保证数据的物理独立性

当数据库的存储构造改变了 [例如选用了另一种存储构造]，数据库管理员修改模式/模式映像，使模式保持不变。应用程序不受影响。保证了数据与程序的物理独立性，简称数据的物理独立性。

优点：

[1] 保证了数据库外模式的稳定性。

[2] 从底层保证了应用程序的稳定性，除非应用需求本身发生变化，否则应用程序一般不需要修改。

[3] 数据与程序之间的独立性，使得数据的定义和描述可以从应用程序中别离出去。

什么叫数据与程序的物理独立性.什么叫数据与程序的逻辑独立性.为什么数据库系统具有数据与程序的独立性.

1、数据与程序的逻辑独立性：当模式改变时，数据库管理员修改有关的外模式/模式映像，使外模式保持不变。从而应用程序不必修改，保证了数据与程序的逻辑独立性，简称数据的逻辑独立性。

2、数据与程序的物理独立性：当数据库的存储构造改变了 [例如选用了另一种存储构造]，数据库管理员修改模式/模式映像，使模式保持不变。应用程序不受影响。保证了数据与程序的物理独立性，简称数据的物理独立性。

数据库管理系统在三级模式之间提供的二层影响保证了数据系统中的数据具有较高的逻辑独立性和物理独立性。

数据库系统的组成 硬件平台及数据库，软件，人员

第二章：关系数据库

关系的完整性约束

实体完整性和参照完整性：关系模型必须满足的完整性约束条件称为关系的两个不变性，应该由关系系统自动

支持。

用户定义的完整性：应用领域需要遵循的约束条件，表达了具体领域中的语义约束。

外码，主码，候选码的概念

候选码：假设关系中的*一属性组的职能唯一地标识一个元组，则称该属性组为候选码。

主码：假设一个关系有多个候选码，则选定期中一个为主码。

外部码：设 F 是根本关系 R 的一个或一组属性。但不是关系 R 的码，如果 F 与根本关系 S 的主码 K 相对应，则称 F 是根本关系 R 的外部码，简称外码。

关系的 3 类完整性约束概念

实体完整性：假设属性〔指一个或一组属性〕A 是根本关系 R 的主属性，A 不能取空值。

参照完整性：假设属性〔或属性组〕F 是根本关系 R 的外码，它是根本关系 S 的主码 K 相对应〔根本关系 R 和 S 不一定是不同的关系〕，则对于 R 中每个元组在 F 上的值必须为：或者取空值〔F 的每个属性值均为空值〕；或者等于 S 中*个元组的主码值。

用户定义的完整性：针对*一具体关系数据库的约束条件。反映*一具体应用所设计的数据必须满足的语义要求。

关系操作的特点，关系代数中的各种运算

关系操作的特点是集合操作方式，即操作的对象和结果是集合。

关系代数 1、并〔 $R \cup S$ 〕 仍为 n 目关系，由属于 R 或属于 S 的元组组成。 $R \cup S = \{t | t \in R \vee t \in S\}$

2、差〔 $R - S$ 〕 仍为 n 目关系，由属于 R 而不属于 S 的所有元组组成。 $R - S = \{t | t \in R \wedge t \notin S\}$

3、交〔 $R \cap S$ 〕 仍为 n 目关系，由既属于 R 又属于 S 的元组组成。 $R \cap S = \{t | t \in R \wedge t \in S\}$ $R \cap S = R - (R - S)$

4、笛卡尔积 $R \times S$ ：n 目关系， k_1 个元组；S: m 目关系， k_2 个元组； $R \times S$ 。

9、选择：选择又称为限制〔Restriction〕 σ ：对元组按照条件进展筛选。在关系 R 中选择满足给定条件的诸元组 $\sigma_F(R) = \{t | t \in R \wedge F(t) = \text{'真'}\}$ 。

10、投影：投影运算符 π 的含义：从 R 中选择出假设干属性列组成新的关系 $\pi_A(R) = \{t[A] | t \in R\}$ A: R 中的属性列投影操作主要是从列的角度进展运算。但投影之后不仅取消了原关系中的*些列，而且还可能取消*些元组〔防止重复行〕。

11、连接：连接也称为 θ 连接：两表中的元组有条件的串接。从两个关系的笛卡尔积中选取属性间满足一定条件的元组 $R \bowtie S = \{ \mid tr \in R \wedge ts \in S \wedge tr[A] \theta ts[B] \}$

外连接：如果把舍弃的元组也保存在结果关系中，而在其他属性上填空值(Null)，这种连接就叫做外连接。

左外连接：如果只把左边关系 R 中要舍弃的元组保存就叫做左外连接(LEFT OUTER JOIN 或 LEFT JOIN)。

右外连接：如果只把右边关系 S 中要舍弃的元组保存就叫做右外连接(RIGHT OUTER JOIN 或 RIGHT JOIN)。

12、除 $\div$ ：给定关系 R(*, Y) 和 S(Y, Z)，其中*, Y, Z 为属性组；R 中的 Y 与 S 中的 Y 可以有不同的属性名，但必须出自一样的域集；R 与 S 的除运算得到一个新的关系 P(*), P 是 R 中满足以下条件的元组在 * 属性列上的投影

第三章：关系数据库标准语言 SQL

注意：SQL(Oracle 除外)一般不提供修改视图定义和索引定义的操作，需要先删除再重建

定义根本表：CREATE TABLE <表名>

〔<列名> <数据类型>〔<列级完整性约束条件>〕

〔, <列名> <数据类型>〔<列级完整性约束条件>〕〕…

〔, <表级完整性约束条件>〕〕；

列级完整性约束--涉及到该表的一个属性

- NOT NULL：非空值约束

- UNIQUE: 唯一性〔单值约束〕约束
- PRIMARY KEY:主码约束
- DEFAULT <默认值>:默认〔缺省〕约束
- Check < (逻辑表达式) >:核查约束,定义校验条件
- NOT NULL : 非空值约束
- UNIQUE: 唯一性〔单值约束〕约束
- PRIMARY KEY:主码约束
- DEFAULT <默认值>:默认〔缺省〕约束
- Check < (逻辑表达式) >:核查约束,定义校验条件

表级完整性约束--涉及到该表的一个或多个属性。

- UNIQUE(属性列列表) : 限定各列取值唯一
- PRIMARY KEY (属性列列表) : 指定主码
- FOREIGN KEY (属性列列表) REFERENCES <表名> [(属性列列表)]
- Check(<逻辑表达式>) : 检查约束
- PRIMARY KEY 与 UNIQUE 的区别.

例: 建立“学生”表 Student, **是主码, 取值唯一

CREATE TABLE Student

(Sno CHAR(9) PRIMARY KEY, /*主码*/

Sname CHAR(20) UNIQUE, /* Sname 取唯一值*/

Sse* CHAR(2), Sage SMALLINT, Sdept CHAR(20));

数据类型

修改根本表: ALTER TABLE <表名>

[ADD <新列名> <数据类型> [完整性约束]]

[DROP <列名> | <完整性约束名>]

[ALTER COLUMN<列名> <数据类型>];

例: 向 Student 表增加“入学时间”列, 其数据类型为日期型

ALTER TABLE Student ADD S_entrance DATE;

不管根本表中原来是否已有数据, 新增加的列一律为空值

将年龄的数据类型由字符型〔假设原来的数据类型是字符型〕改为整数

ALTER TABLE Student ALTER COLUMN Sage INT;

注: 修改原有的列定义有可能会破坏已有数据

增加课程名称必须取唯一值的约束条件。

ALTER TABLE Course ADD UNIQUE(ame);

直接删除属性列:(新标准) 例: ALTER TABLE Student Drop Sage;

删除根本表 : DROP TABLE <表名> [RESTRICT| CASCADE];

RESTRICT: (受限)欲删除的根本表不能被其他表的约束所引用, 如果存在依赖该表的对象〔触发器, 视图等〕, 则此表不能被删除。

CASCADE: [级联] 在删除根本表的同时, 相关的依赖对象一起删除。

例: 删除 Student 表 DROP TABLE Student CASCADE ;

根本表定义被删除, 数据被删除; 表上建立的索引、视图、触发器等一般也将被删除。

删除索引 : DROP INDE* <索引名>;

删除索引时, 系统会从数据字典中删去有关该索引的描述。

例: 删除 Student 表的 Stusname 索引: DROP INDE* Stusname

2、数据查询: 根本格式

单表查询

选择表中的假设干列〔投影〕

查询指定列 [相当于 $\pi A(R)$, $A = A_1, A_2, \dots, A_n$]

例：查询全体学生的**与

```
SELECT Sno, Sname
```

```
FROM Student;
```

查询全部列：在 SELECT 关键字后面列出所有列名按用户指定顺序显示。

将<目标列表表达式>指定为 *按关系模式中的属性顺序显示。

例：查询全体学生的详细记录

```
SELECT Sno, Sname, Sse*, Sdept, Sage
```

```
FROM Student;
```

或

```
SELECT *
```

```
FROM Student;
```

P.S: SELECT 子句的<目标列表表达式>可以为：

查询经过计算的值

例：查全体学生的及其出生年份

```
SELECT Sname, 2021-Sage /*假定当年的年份为 2021 年*/
```

```
FROM Student;
```

输出结果：

```
Sname    2021-Sage
```

```
勇        1991
```

```
晨        1992
```

字符串常量、函数

例：查询全体学生的、出生年份和所有系，要求用小写字母

表示所有系名

```
SELECT Sname, 'Year of Birth: ', 2004-Sage, ISLOWER(Sdept)
```

```
FROM Student;
```

输出结果：

```
Sname    'Year of Birth:'  2004-Sage    ISLOWER(Sdept)
```

```
勇    Year of Birth:    1984        cs
```

```
晨    Year of Birth:    1985        is
```

列别名

```
SELECT Sname as NAME, 'Year of Birth: ' as BIRTH,
```

```
2021-Sage as BIRTHDAY, LOWER(Sdept) as DEPARTMENT
```

```
FROM Student;
```

输出结果：

```
NAME      BIRTH      BIRTHDAY      DEPARTMENT
```

```
-----
```

```
勇    Year of Birth:    1991        cs
```

```
晨    Year of Birth:    1992        is
```

选择表中的假设元组 [选择]

消除重复性：

指定 DISTINCT 关键词，去掉表中重复的行

```
SELECT DISTINCT Sno FROM SC;
```

注意 DISTINCT 短语的作用围是所有目标列

错误的写法

```
SELECT DISTINCT o, DISTINCT Grade FROM SC;
```

正确的写法

```
SELECT DISTINCT o, Grade FROM SC;
```

SELECT 子句缺省情况是保存重复元组(ALL),

例：查询选修了课程的学生**。SELECT Sno FROM SC; 等价于：

SELECT ALL Sno FROM SC;

查询满足条件的元组

WHERE 子句常用的查询条件 [相当于oF]

比拟大小使用比拟运算符 或逻辑运算符 NOT + 比拟运算符

例：查询计算机科学系全体学生的

SELECT Sname

FROM Student

WHERE Sdept= 'CS';

例：查询所有年龄在 20 岁以下的学生及其年龄

SELECT Sname, Sage

FROM Student

WHERE Sage < 20; //NOT Sage>=20

确定范围 BETWEEN ... AND ...NOT BETWEEN ... AND ...

例：查询年龄在 20~23 岁 [包括 20 岁和 23 岁] 之间的学生的、系别和年龄

SELECT Sname, Sdept, Sage

FROM Student

WHERE Sage BETWEEN 20 AND 23; //Sage>=20 and Sage<=23

例：查询年龄不在 20~23 岁之间的学生、系别和年龄

SELECT Sname, Sdept, Sage

FROM Student

WHERE Sage NOT BETWEEN 20 AND 23; //Sage<20 or Sage>23

确定集合 IN <值表>, NOT IN <值表>

例：查询信息系 [IS]、数学系 [MA] 和计算机科学系 [CS] 学生的和性别

SELECT Sname, Ssex

FROM Student

WHERE Sdept IN ('IS', 'MA', 'CS');

例：查询既不是信息系、数学系，也不是计算机科学系的学生和性别

SELECT Sname, Ssex

FROM Student

WHERE Sdept NOT IN ('IS', 'MA', 'CS');

自左匹配[NOT] LIKE '<匹配串>' [ESCAPE '<换码字符>']

匹配串为固定字符串

例：查询**为 200215121 的学生的详细情况。

SELECT * SELECT *

FROM Student 等价于： FROM Student

WHERE Sno LIKE '200215121'; WHERE Sno = '200215121';

匹配串为含通配符的字符串

%: 代表任意长度 [可以是 0] 的字符串

_ : 代表任意单个字符

字符串本身就含有 % 或 _ 时，在 % 或 _ 之前加上转义符 “\” 要使用 ESCAPE '<换码字符>' 将通配符转义为普通字符。如果 ‘\’ 要作为一个普通字符，用连续两个 ‘\’ 表示一个真正的 ‘\’。

例：查询以 "DB_" 开头，且倒数第 3 个字符为 i 的课程详细情况。

SELECT *

FROM Course

WHERE name LIKE 'DB__%i_ ' ESCAPE ' \ ' ;

涉及空值的查询 IS NULL 或 IS NOT NULL “IS” 不能用 “=” 代替

例：查所有有成绩的学生**和课程号

```
SELECT Sno, o
```

```
FROM SC
```

```
WHERE Grade IS NOT NULL;
```

多重条件查询：AND 和 OR 来联结多个查询条件，AND 的优先级高于 OR，可以用括号改变优先级，可用来

实

现多种其他谓词。

[NOT] IN

[NOT] BETWEEN ... AND ...

改写 例：查询信息系 [IS]、数学系 [MA] 和计算机科学系 [CS] 学生的和性别

```
SELECT Sname, Sse*
```

```
FROM Student
```

```
WHERE Sdept IN ( 'IS', 'MA', 'CS' )
```

可改写为：

```
SELECT Sname, Sse*
```

```
FROM Student
```

```
WHERE Sdept= ' IS ' OR Sdept= ' MA' OR Sdept= ' CS '
```

ORDER BY 子句：对查询结果排序

可以按一个或多个属性列排序：升序：ASC；降序：DESC；缺省值为升序。

当排序列含空值时：空值最大

ASC：排序列为空值的元组最后显示；DESC：排序列为空值的元组最先显示。

例：查询选修了 3 号课程的学生的**及其成绩，查询结果按分数降序排列

```
SELECT Sno, Grade
```

```
FROM SC
```

```
WHERE o= ' 3 '
```

```
ORDER BY Grade DESC;
```

聚集函数：对查询结果集中的*列进展计算或统计。

计数 COUNT [(DISTINCT|ALL) *] COUNT [(DISTINCT|ALL) <列名>]

计算总和 SUM [(DISTINCT|ALL) <列名>]

计算平均值 AVG [(DISTINCT|ALL) <列名>]

最大最小值 MAX [(DISTINCT|ALL) <列名>] MIN [(DISTINCT|ALL) <列名>]

例：查询学生 200215012 选修课程的总学分数

```
SELECT SUM(Ccredit)
```

```
FROM SC, Course
```

```
WHERE Sno='200215012' AND SC.o=Course.o;
```

注：除 Count [*] ,都要跳过空值；Where 子句不能使用聚集函数。

GROUP BY 子句：对查询结果分组。

用途细化聚集函数的作用对象

未对查询结果分组，聚集函数将作用于整个查询结果

对查询结果分组后，聚集函数将分别作用于每个组

使用 GROUP BY 后：其 SELECT 子句的列名列表中只能出现分组属性和集函数。

如果分组后还要按照条件对这些组进展筛选，可使用 having 短语指定筛选条件

例： 查询选修了 3 门以上课程的学生**

```
SELECT Sno
```

```
FROM SC
```

GROUP BY Sno

HAVING COUNT(*) >3;

例：查询有 3 门以上课程是 90 分以上学生的**及 [90 分以上的] 课程数

SELECT Sno, COUNT(*)

FROM SC

Where Grade >90

GROUP BY Sno

HAVING COUNT(*) >3;

HAVING 短语与 WHERE 子句的区别：作用对象不同

WHERE 子句作用于基表或视图，从中选择满足条件的元组

HAVING 短语作用于组，从中选择满足条件的组。

****连接查询****p100 页

****嵌套查询****p104 页 ***重点***

数据更新

[1] 插入数据

插入元组

INSERT

[INTO] <表名> [(<属性列 1>[, <属性列 2>...])]

VALUES (<常量 1> [, <常量 2>] ...)

功能：将新元组插入指定表中；新元组的属性列 1 的值为常量 1，属性列 2 的值为常量 2，…。

INTO 子句：属性列的顺序可与表定义中的顺序不一致，但须指定列名；没有指定属性列，表示要插入的是一条完整的元组；指定局部属性列，未指定的属性列取空值，具有 NOT NULL 的属性列除外。

VALUES 子句：提供的值必须与 INTO 子句匹配，值的个数，值的类型。

例：将一个新学生元组 [**： 200215128； 冬； 性别： 男； 所在系： IS； 年龄： 18 岁] 插入到 Student 表中

INSERT

INTO Student (Sno, Sname, Sse*, Sdept, Sage)

VALUES ('200215128', '冬', '男', 'IS', 18);

例： 将学生成民的信息插入到 Student 表中

INSERT

INTO Student

VALUES ('200215126', '成民', '男', 18, 'CS');

例：插入一条选课记录('200215128', '1 ')。

INSERT

INTO SC(Sno, o)

VALUES (' 200215128 ', ' 1 ');

RDBMS 将在新插入记录的 Grade 列上自动地赋空值。或者：

INSERT

INTO SC

VALUES (' 200215128 ', ' 1 ', NULL);

因为没有指出 SC 的属性名，在 GRADE 列上要明确给出空值

插入子查询结果

Insert

Into <表名>[(<属性列 1>[, <属性列 2>...)]

子查询 [select 等];

[2] 修改数据

UPDATE <表名>

SET <列名>=<表达式>[, <列名>=<表达式>]...

[WHERE <条件>];

SET 子句: 指定修改方式, 要修改的列, 修改后取值: <表达式>。

WHERE 子句: 指定要修改的元组, 缺省表示要修改表中的所有元组。

功能: 修改指定表中满足 WHERE 子句条件的元组。

修改*一个元组的值

例: 将学生 200215121 的年龄改为 22 岁

```
UPDATE Student
```

```
SET Sage=22
```

```
WHERE Sno='200215121';
```

修改多个元组的值

例: 将所有学生的年龄增加 1 岁

```
UPDATE Student
```

```
SET Sage= Sage+1;
```

带子查询的修改语句

子查询须放在比拟运算符之后

例: 将计算机科学系全体学生的成绩置零。

```
UPDATE SC
```

```
SET Grade=0
```

```
WHERE 'CS' =
```

```
(SELETE Sdept
```

```
FROM Student
```

```
WHERE Student.Sno = SC.Sno);
```

[3] 删除数据

DELETE

FROM <表名>

[WHERE <条件>];

功能: 删除指定表中满足 WHERE 子句条件的元组。

WHERE 子句: 指定要删除的元组; 缺省表示要删除表中的全部元组, 表的定义仍在数据字典中。

删除*一个元组的值

例: 删除**为 200215128 的学生记录

```
DELETE
```

```
FROM Student
```

```
WHERE Sno= 200215128';
```

删除多个元组的值

例: 删除所有的学生选课记录

```
DELETE
```

```
FROM SC;
```

带子查询的删除语句

例：删除计算机科学系所有学生的选课记录

```
DELETE
FROM SC
WHERE 'CS'=
(SELETE Sdept
FROM Student
WHERE Student.Sno=SC.Sno);
```

4、视 图

特点：虚表，是从一个或几个根本表〔或视图〕导出的表；只存放视图的定义，不存放视图对应的数据；基表中的数据发生变化，从视图中查询出的数据也随之改变。

基于视图的操作： 查询、删除、受限更新、定义基于该视图的新视图。

〔1〕定义视图

建立视图

```
CREATE VIEW <视图名> [(<列名> [, <列名>]...)]
AS <子查询>
[WITH CHECK OPTION];
```

子查询：不允许含有 ORDER BY 子句和 DISTINCT 短语。

WITH CHECK OPTION：表示对视图进展 UPDATE，INSERT 和 DELETE 操作时要保证更新、插入或删除的行满足视图定义中的谓词条件〔即子查询中的条件表达式〕。

组成视图的属性列名：全部省略或全部指定，但在以下三种情况下必须明确指定组成视图的所有列名：

*个目标列不是单纯的属性名，而是聚集函数或列表表达式；

多表连接时选出了几个同名列作为视图的字段；

需要在视图中为*个列启用新的名字。

RDBMS 执行 CREATE VIEW 语句时只是把视图定义存入数据字典，并不执行其中的 SELECT 语句。在对视图查询时，按视图的定义从根本表中将数据查出。

行列子集视图：从单个根本表导出，只是去掉了根本表的*些行和*些列保存了主码

例：建立信息系学生的视图

```
CREATE VIEW IS_Student
AS
```

```
SELECT Sno, Sname, Sage
FROM Student
WHERE Sdept= 'IS' ;
```

WITH CHECK OPTION 例：建立信息系学生的视图，并要求进展修改和插入操作时仍需保证该视图只有信息系的学生

```
CREATE VIEW IS_Student
AS
SELECT Sno, Sname, Sage
FROM Student
WHERE Sdept= 'IS'
WITH CHECK OPTION;
```

加上了 WITH CHECK OPTION 子句：RDBMS 对 IS_Student 视图的更新操作：修改操作：自动加上 Sdept='IS' 的条件；删除操作：自动加上 Sdept='IS' 的条件；插入操作：自动检查 Sdept 属性值是否为 'IS'。如果不是，则拒绝该插入操作。如果没有提供 Sdept 属性值，则自动定义 Sdept 为 'IS'。

基于多个基表的视图

例：建立信息系选修了 1 号课程的学生视图

```
CREATE VIEW IS_S1(Sno, Sname, Grade)
AS
SELECT Student.Sno, Sname, Grade
FROM Student, SC
WHERE Sdept= 'IS' AND
Student.Sno=SC.Sno AND
SC.o= '1';
```

基于视图的视图

例：建立信息系选修了 1 号课程且成绩在 90 分以上的学生的视图

```
CREATE VIEW IS_S2
AS
SELECT Sno, Sname, Grade
FROM IS_S1
WHERE Grade>=90;
```

由于视图中的数据不会实际存储，所以定义视图时可根据应用的需要，设置一些派生属性列或虚拟列，以便于查询和统计。

以 SELECT * 方式创立的视图可扩大性差，应尽可能防止。

缺点：修改基表 Student 的构造后，Student 表与 F_Student 视图的映象关系被破坏，导致该视图不能正常工作。

删除视图 DROP VIEW <视图名>;

该语句从数据字典中删除指定的视图定义。如果该视图上还导出了其他视图，使用 CASCADE 级联删除语句，把该视图和由它导出的所有视图一起删除。删除基表时，由该基表导出的所有视图定义都必须显式地使用 DROP VIEW 语句删除。

例：删除视图 BT_S:

```
DROP VIEW BT_S;
```

删除视图 IS_S1: 拒绝执行

级联删除: DROP VIEW IS_S1 CASCADE;

〔2〕 查询视图：查询视图与查询根本表一样,视图定义后，就可以像对待根本表一样对视图进展查询〔SELECT〕操作。

视图消解法 [View Resolution] 进展有效性检查，检查查询的表、视图等是否存在。如果存在，则从数据字典中取出视图的定义；转换成等价的对根本表的查询，把视图定义中的子查询与用户的查询结合起来；执行修正后的查询。

例：在信息系学生的视图中找出年龄小于 20 岁的学生

```
SELECT Sno, Sage
FROM IS_Student
WHERE Sage<20;
```

视图消解转换后的查询语句为：

```
SELECT Sno, Sage
FROM Student
WHERE Sdept= 'IS' AND Sage<20;
```

例：在 S_G 视图中查询平均成绩在 90 分以上的学生**和平均成绩

```
SELECT *
```

```

FROM S_G
WHERE Gavg>=90;
S_G 视图的子查询定义:
CREATE VIEW S_G (Sno, Gavg)
AS
SELECT Sno, AVG(Grade)
FROM SC
GROUP BY Sno;

```

[3] 更新视图

DBMS 实现视图更新的方法：转换为对根本表的更新。视图消解法 [View Resolution]。

只有对成为“可更新”视图才能进展更新操作。

SQL2 对“可更新”视图给出正式定义：从关系 R 选出*些属性〔用 select 而不是 select distinct〕定义的视图，R 本身可以是可更新的视图；Where 中不能嵌套涉及 R 的子查询；Select 必须包括足够多的属性，所有 not null 的属性必须包括。

对于视图元组的更新操作 [INSERT、DELETE、UPDATA]，有以下三条规则：

如果一个视图是从多个根本表使用联接操作导出的，则不允许对这个视图执行更新操作。

如果在导出视图的过程中，使用了分组和聚集函数操作，也不允许对这个视图执行更新操作。

行列子集视图可以执行更新操作。

在 SQL2 中，允许更新的视图在定义时，必须加上“WITH CHECK OPTION”短语。DBMS 在更新视图时会进展检查，防止用户通过视图对不属于视图围的根本表数据进展更新。

例：将信息系学生视图 IS_Student 中**200215122 的学生改为“辰”。

```

UPDATE IS_Student
SET Sname= '辰'
WHERE Sno= '200215122';
转换后的语句:
UPDATE Student
SET Sname= '辰'
WHERE Sno= '200215122' AND Sdept= 'IS';

```

例：向信息系学生视图 IS_S 中插入一个新的学生记录：200215129，新，20 岁

```

INSERT
INTO IS_Student
VALUES( '95029' , '新' , 20);
转换为对根本表的更新:
INSERT
INTO Student(Sno, Sname, Sage, Sdept)
VALUES( '200215129 ', '新', 20, 'IS' );

```

例：删除信息系学生视图 IS_Student 中**为 200215129 的记录

```

DELETE
FROM IS_Student
WHERE Sno= '200215129';
转换为对根本表的更新:
DELETE
FROM Student

```

WHERE Sno= ' 200215129 ' AND Sdept= 'IS';

更新视图的限制：一些视图是不可更新的，因为对这些视图的更新不能唯一地有意义地转换成对相应根本表的更新。对其他类型视图的更新不同系统有不同限制：

- (1) 假设视图是由两个以上根本表导出的，则此视图不允许更新
- (2) 假设视图的字段来自字段表达式或常数，则不允许对此视图执行 INSERT 和 UPDATE 操作，但允许执行 DELETE 操作。
- (3) 假设视图的字段来自集函数，则此视图不允许更新。
- (4) 假设视图定义中含有 GROUP BY 子句，则此视图不允许更新。
- (5) 假设视图定义中含有 DISTINCT 短语，则此视图不允许更新。
- (6) 假设视图定义中有嵌套查询，并且层查询的 FROM 子句中涉及的表也是导出该视图的根本表，则此视图不允许更新。
- (7) 一个不允许更新的视图上定义的视图也不允许更新

第四章：数据库的平安性

计算机系统的三类平安性问题

技术平安，管理平安，政策法律

数据库平安性控制〔6种〕

1.用户标识与鉴别 用户标识，口令

2.存取控制

3.自主存取控制方法：定义各个用户对不同数据对象的存取权限。当用户要数据库时，首先要检查其存取权限，以防止非法用户对数据库进展存取。“自主存取控制”中“自主”的含义：用户可以将自己所拥有的存取权限“自主”地授予他人，即用户具有一定的“自主”权。

4.授权与收回语句。

GRANT 语句和 REVOKE 语句实现关系数据库系统中存取控制权限：

1、GRANT〔授权〕

GRANT 语句的一般格式：

GRANT <权限>[,<权限>]...

ON <对象类型> <对象名> , [<对象类型> <对象名>]

TO <用户>[,<用户>]...

[WITH GRANT OPTION];

将对指定操作对象的指定操作权限授予指定的用户

发出 GRANT：DBA，数据库对象创立者〔即属主 Owner〕，拥有该权限的用户

承受权限的用户：一个或多个具体用户；PUBLIC〔全体用户〕。

例：把查询 Student 表权限授给用户 U1

GRANT SELECT

ON TABLE Student

TO U1;

例：把查询 Student 表和修改学生**的权限授给用户 U4

GRANT UPDATE(Sno), SELECT

ON TABLE Student

TO U4;

对属性列的授权时必须明确指出相应属性列名

2、REVOKE：授予的权限可以由 DBA 或其他授权者用 REVOKE 语句收回

REVOKE 语句的一般格式为：

REVOKE <权限>[,<权限>]...

ON <对象类型> <对象名> >[,<对象类型> <对象名>] ...

FROM <用户>[,<用户>]... >[CASCADE|RESTRICT];

把指定对象的指定操作权限从指定用户处收回。

例：把用户 U4 修改学生**的权限收回

REVOKE UPDATE(Sno)

ON TABLE Student

FROM U4;

例：收回所有用户对表 SC 的查询权限

REVOKE SELECT

ON TABLE SC

FROM PUBLIC;

例：把用户 U5 对 SC 表的 INSERT 权限收回

REVOKE INSERT

ON TABLE SC

FROM U5 CASCADE ; [缺省是 RESTRICT]

将用户 U5 的 INSERT 权限收回的时候必须级联 [CASCADE] 收回，因为 U5 将 SC 的 INSERT 权限授予 U6，U6 又将其授予 U7，CASCADE 系统只收回直接或间接从 U5 处获得的权限。

Grant 和 Revoke 向用户授予或收回对数据的操作权限

3、创立数据库模式的权限：DBA 在创立用户时实现

CREATE USER 语句格式

CREATE USER <username>

[WITH] [DBA | RESOURCE | CONNECT]

拥有 DBA 权限的用户是系统中的超级用户；只有系统的超级用户才有权创立新的数据库用户；如果没有指定创立的新用户的权限，默认该用户拥有 CONNECT 权限,只能登录数据库。

5.数据库角色 角色的创立，给角色授权，将一个角色授予其他角色或用户

6.强制存取控制方法：每一个数据对象被 [强制地] 标以一定的加别，每位用户也被 [强制地] 授予*一级别的许可证。系统规定只有具有*一许可证级别的用户才能存取加别的数据对象。

强制存取控制 [MAC] 是对数据本身进展标记，无论数据如何复制，标记与数据是一个不可分的整体，只有符合标记要求的用户才可以操纵数据，从而提高了更高级别的平安性。

第五章：数据库的完整性 看书。。。

实体完整性 [定义] 参照完整性 用户定义完整性

第六章：关系数据理论

1、函数依赖：设 $R(U)$ 是一个属性集 U 上的关系模式， $*$ 和 Y 是 U 的子集，假设对于 $R(U)$ 的任意一个可能的关系 r ， r 中不可能存在两个元组在 $*$ 上的属性值相等，而在 Y 上的属性值不等，则称 “ $*$ 函数确定 Y ” 或 “ Y 函数依赖于 $*$ ”，记作 $* \rightarrow Y$ 。

所谓函数依赖是指关系中属性或属性组的值可以决定其它属性的值，设 $R(U)$ 是属性集 U 上的关系模式， $*$ 、 Y 是 U 的子集：

如果 $*$ 和 Y 之间是 1:1 关系 [一对一关系]，如学校和校长之间就是 1:1 关系，则存在函数依赖 $* \rightarrow Y$ 和 $Y \rightarrow *$ 。

如果 $*$ 和 Y 之间是 1:n 关系 [一对多关系]，如年龄和之间就是 1:n 关系，则存在函数依赖 $Y \rightarrow *$ 。

如果 $*$ 和 Y 之间是 m:n 关系 [多对多关系]，如学生和课程之间就是 m:n 关系，则 $*$ 和 Y 之间不存在函数依赖。

在关系模式 $R(U)$ 中，对于 U 的子集 $*$ 和 Y ，如果 $* \rightarrow Y$ ，但 $Y \not\rightarrow *$ ，则称 $* \rightarrow Y$ 是非平凡的函数依赖；假设 $* \rightarrow Y$ ，但 $Y \rightarrow *$ ，则称 $* \rightarrow Y$ 是平凡的函数依赖。

例：在关系 $SC(Sno, o, Grade)$ 中，非平凡函数依赖： $(Sno, o) \rightarrow Grade$;

平凡函数依赖: $(Sno, o) \rightarrow Sno$ $(Sno, o) \rightarrow o$

假设 $* \rightarrow Y$, 则 $*$ 称为这个函数依赖的决定属性组, 也称为决定因素 [Determinant]。

2、完全函数依赖: 在 $R(U)$ 中, 如果 $* \rightarrow Y$, 并且对于 $*$ 的任何一个真子集 $*$ ' , 都有 $*$ ' $\not\rightarrow Y$, 则称 Y 对 $*$ 完

全函数依赖, 记作 $X \xrightarrow{F} Y$ 。

3、局部函数依赖: 假设 $* \rightarrow Y$, 但 Y 不完全函数依赖于 $*$, 则称 Y 对 $*$ 局部函数依赖, 记作 $X \xrightarrow{P} Y$

例: 中 $(Sno, o) \rightarrow Grade$ 是完全函数依赖, $(Sno, o) \rightarrow Sdept$ 是局部函数依赖

$\because Sno \rightarrow Sdept$ 成立, 且 Sno 是 $[Sno, o]$ 的真子集。

当存在局部依赖时, 就会产生数据冗余。

4、传递函数依赖: 在 $R(U)$ 中, 如果 $* \rightarrow Y, (Y \not\rightarrow *) , Y \rightarrow Z, Z \not\rightarrow Y$, 则称 Z 对 $*$ 传递函数依赖, 记为: $X \xrightarrow{传递} Z$

注: 如果 $Y \rightarrow *$, 即 $* \leftrightarrow Y$, 则 Z 直接依赖于 $*$ 。

例: 在关系 $Std(Sno, Sname, Sdept, Mname)$ 中, 有:

$Sno \rightarrow Sdept, Sdept \rightarrow Mname$

$Mname$ 传递函数依赖于 Sno

5、候选码: 设 K 为 $R<U, F>$ 中的属性或属性组, 假设 $KK \xrightarrow{F} U$ [每个属性], 则 K 称为 R 的候选码 [Candidate Key]

注 K 满足两个条件:

1. K 完全函数决定该关系的所有其它属性。

2. K 的任何真子集都不能完全函数决定 R 的所有其它属性, K 必须是最小的。

假设候选码多于一个, 则选定其中的一个做为主码 [Primary Key], 通常称之为码。

主属性 [Prime attribute]: 包含在任何一个候选码中的属性。

非主属性或非码属性: 不包含在任何码中的属性。

例: 关系模式 $S(Sno, Sdept, Sage)$, 单个属性 Sno 是码,

$SC [Sno, o, Grade]$ 中, $[Sno, o]$ 是码

由于码能唯一确定一个元组, 所以关系的码函数决定该关系的所有属; 一个关系中的所有属性都函数依赖于该关系的码。

例: 关系模式 $R [P, W, A]$ P : 演奏者 W : 作品 A : 听众: 一个演奏者可以演奏多个作品, $*$ 一作品可被多个演奏者演奏, 听众可以欣赏不同演奏者的不同作品。

码为 (P, W, A) , 即 All-Key

6、外部码: 关系模式 R 中属性或属性组 $*$ 并非 R 的码, 但 $*$ 是另一个关系模式的码, 则称 $*$ 是 R 的外部码 [Foreign key] 也称外码

如在 $SC [Sno, o, Grade]$ 中, Sno 不是码,

但 Sno 是关系模式 $S [Sno, Sdept, Sage]$ 的码, 则 Sno 是关系模式 SC 的外部码

主码与外部码一起提供了表示关系之间联系的手段

各种式之间存在联系:

1、1NF: 如果一个关系模式 R 的所有属性都是不可分的根本数据项, 则 $R \in 1NF$

第一式是对关系模式的最起码的要求。不满足第一式的数据库模式不能称为关系数据库; 简而言之, 第一式就是无重复的列, 关系数据库研究的关系都是规化的关系。但是满足第一式的关系模式并不一定是一个好的关系模式。

例: 关系模式 $S-L-C(Sno, Sdept, Sloc, o, Grade)$ $Sloc$ 为学生住处, 假设每个系的学生住在同一个地方。

函数依赖包括: $(Sno, o) F Grade$

$(Sno, o) P Sdept \because Sno \rightarrow Sdept$

$(Sno, o) P Sloc \because Sdept \text{ 传递 } Sloc$

$S-L-C$ 的码为 (Sno, o) 。

S-L-C 满足第一式。

非主属性 Sdept 和 Sloc 局部函数依赖于码(Sno, o)。

插入异常：如未选课的学生不能插入。

删除异常：如放弃修一门课，只选修这门课的学生被删除

数据冗余度大修改复杂

原因：存在对码的冗余依赖。Sdept、Sloc 局部函数依赖于码。

解决方法：S-L-C 分解为两个关系模式，以消除这些局部函数依赖

SC [Sno, o, Grade] S-L [Sno, Sdept, Sloc]

2、2NF；假设 $R \in 1NF$ ，且每一个非主属性完全函数依赖于码，则 $R \in 2NF$

简而言之，第二式就是每一行被码唯一标识

例：S-L-C(Sno, Sdept, Sloc, o, Grade) $\in 1NF$

S-L-C(Sno, Sdept, Sloc, o, Grade) $\in 2NF$

SC [Sno, o, Grade] $\in 2NF$

S-L [Sno, Sdept, Sloc] $\in 2NF$

采用投影分解法将一个 1NF 的关系分解为多个 2NF 的关系，可以在一定程度上减轻原 1NF 关系中存在的插入异常、删除异常、数据冗余度大、修改复杂等问题。但将一个 1NF 关系分解为多个 2NF 的关系，并不能完全消除关系模式中的各种异常情况和数据冗余。

3、3NF：关系模式 $R<U, F>$ 中假设不存在这样的码*、属性组 Y 及非主属性 Z [Z $\rightarrow$ Y]，使得 $* \rightarrow Y$ ， $Y \rightarrow *$ ， $Y \rightarrow Z$ 成立，即每个非主属性都不传递依赖于 R 的码，则称 $R<U, F> \in 3NF$ 。假设 $R \in 3NF$ ，则每一个非主属性既不局部依赖于码也不传递依赖于码。

简而言之，第三式 [3NF] 要求一个数据库表中不能包含其它表中已包含的非码信息。

例：S-L(Sno, Sdept, Sloc) $\in 2NF$ S-L(Sno, Sdept, Sloc) $\in 3NF$

S-D(Sno, Sdept) $\in 3NF$ D-L(Sdept, Sloc) $\in 3NF$

如果 $R \in 3NF$ ，则 R 也是 2NF。

局部依赖和传递依赖是模式产生数据冗余和操作异常的两个重要原因。

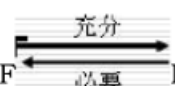
由于 3NF 模式中不存在非主属性对候选码的局部依赖和传递依赖，因此一定程度上解决原 2NF 关系中存在的插入异常、删除异常、数据冗余度大、修改复杂等问题具有较好的性能。将一个 2NF 关系分解为多个 3NF 的关系后，仍然不能完全消除关系模式中的各种异常情况和数据冗余。

4、BC 式 [BF]：关系模式 $R<U, F> \in 1NF$ ，假设 $* \rightarrow Y$ 且 $Y \rightarrow *$ 时*必含有码，即每个属性都不传递依赖于 R 的码，则 $R<U, F> \in BF$ 。

等价于：每一个决定因素都包含码，即消除任何属性对码的局部和传递函数依赖

假设 $R \in BF$ 所有非主属性对每一个码都是完全函数依赖；所有的主属性对每一个不包含它的码，也是完全函数依赖；没有任何属性完全函数依赖于非码属性。


 $R \in BF \xrightarrow{\text{充分}} R \in 3NF$ 。如果 $R \in 3NF$ ，且 R 只有一个候选码


 $R \in BF \xleftrightarrow{\text{必要}} R \in 3NF$

例：关系模式 S [Sno, Sname, Sdept, Sage] 假定 S 有两个码 Sno, Sname

$S \in 3NF$ $S \in BF$

例：系模式 SJP [S, J, P] s 学生, J 课程, P 名次

函数依赖：[S, J] $\rightarrow$ P; [J, P] $\rightarrow$ S [S, J] 与 [J, P] 都可以作为候选码,属性相交

SJP $\in 3NF$, (不存在非主属性对码的局部和传递依赖)

SJP $\in$ BF,(每个决定因素都包含码)

第七章：数据库设计

三分技术，七分管理，十二分根底数据是数据库设计的特点之一。

数据库设计的根本步骤

- 1、需求分析
- 2、概念结构构造设计
- 3、逻辑构造设计
- 4、物理构造设计
- 5、数据库实施
- 6、数据库运行和维护

数据字典 p207 页

什么是数据库的逻辑构造设计.试述其设计步骤.E-R 图向关系模型的转换

逻辑构造设计 P224

4、数据库的再组织和重构

第十章：数据库恢复技术

事务事务是用户所定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位。事务具有四个特性：

- 1、原子性：事务是数据库的逻辑单位，事务中所包括的读项操作要么都做，要么都不做。
- 2、一致性：事务执行的结果必须是使数据库从一个一致性状态转变到另一个一致性状态。
- 3、隔离性：一个事务的执行不能被其他事务干扰，即一个事务部的操作及所试用的数据对其他并发事务是隔离的，并发执行的各个事务之间不能互相干扰。
- 4、持续性：〔永久性〕事务一旦提交，它对数据库中主句的改变就应该是永久的。接下来的其他操作或不应对其执行结果产生任何影响。

为了保证事务的原子性、一致性与持续性。DBMS 必须对事务故障、系统故障和介质故障进展恢复；为了保证事务的隔离性和一致性，DBMS 需要对并发操作进展控制。

故障的种类及其影响以及相应的恢复策略

1、事务部故障的恢复：事务部故障的恢复由 DBMS 自动完成，对用户而言是透明的。DBMS 执行的恢复步骤如下：

- 〔1〕反向扫描文件日志〔即从后向前扫描日志文件〕，查找该事务的更新操作。
- 〔2〕对该事务的更新操作执行逆操作，即将日志记录中“更新前的值”写入数据库。
- 〔3〕继续反向扫描日志文件，进展同样的处理。
- 〔4〕如此继续下去，直至独到此事务的开场标记，该事务故障恢复就完成了。

2、系统故障的恢复：会造成数据库处于不一致的状态，主要是一方面，为完成事务对数据库所做的更新可能已写入数据库；另一方面，已提交事务对数据库做的更新可能尚留在缓冲区，未能及时写入数据库。因此恢复操作就是撤销〔UNDO〕故障发生时为完成的事务，重做〔REDO〕已完成的事务。恢复步骤如下：

〔1〕正向扫描日志文件，找出在故障发生之前已经提交的事务队列〔REDO 队列〕和为完成的事务队列〔UNDO 队列〕。

〔2〕对于撤销队列中的各个事务进展 UNDO 处理。进展 UNDO 处理的方法是：反向扫描日志文件，对每个 UNDO 事务的过呢更新操作执行逆操作，即将日志记录中“更新前的值”写入数据库中。

〔3〕对重做队列中的各个事务进展 REDO 处理。进展 REDO 处理的方法是：正向扫描日志文件，对每个 REDO

事务重新执行日志文件中所登记的操作，并将日志记录中“更新后的值”写入数据库。

3、截至故障的恢复：恢复方法是重装数据库，然后重做已完成的事务，具体操作如下：

〔1〕 DBA 装入最新的数据库后备副本〔离故障发生时刻最近的转储副本〕,使数据库恢复到转储时的一致性状态。

〔2〕 DBA 装入转储完毕时的日志文件副本。

〔3〕 DBA 启动系统恢复命令，有 DBMS 实现恢复功能，即重做已完成的事务。

恢复的实现技术

数据转储〔动态海量转储，动态增量转储，静态海量转储，静态增量转储〕

登记日志文件

第十一章：数据库并发控制技术

并发操作可能产生的数据不一致

1、丧失修改：两个事务 1、2 同时读入同意数据并进展修改，2 所提交的结果破坏〔覆盖〕了 1 提交的结果，导致 1 所做的修改被丧失。

2、不可重复读：事务 1 读取*一数据后，事务 2 对其执行更新操作，使 1 无法再现前一次读取的结果。

3、读‘脏’数据：事务 1 修改*一数据，将其协会磁盘，事务 2 读取同意数据后，事务 1 由于*种原因被撤销，这是事务 1 已修改正的数据将恢复原值，事务 2 所读取的数据就与数据库中的数据不一致，则事务 2 所独到的就是‘脏’数据，即不正确的数据。

防止不一致性的方法是并发控制机制。最常用的并发控制技术是封锁技术。

封锁 排它锁，共享锁

活锁和死锁 P296

并发调度的可串行性,冲突可串行性化调度 P299

封锁的粒度，多粒度封锁协议 P302